

Patuakhali Science and Technology University

Faculty of Computer Science and Engineering

CIT 412: Compiler Design & Automata Theory Sessional

Lab Report

Submission Date: 11, Nov 2026

Submitted from	Submitted to
Prosenjit Mondol	Dr. Md. Abdul Masud
ID: 2102049	Professor
Reg: 10176	Department of Computer Science and Information Technology,
	Faculty of Computer Science and Engineering

1. Abstract Syntax Tree:

An Abstract Syntax Tree (AST) is a tree representation of an expression in which internal nodes represent operators and leaf nodes represent operands. It shows the logical structure of an expression while removing unnecessary syntax details.

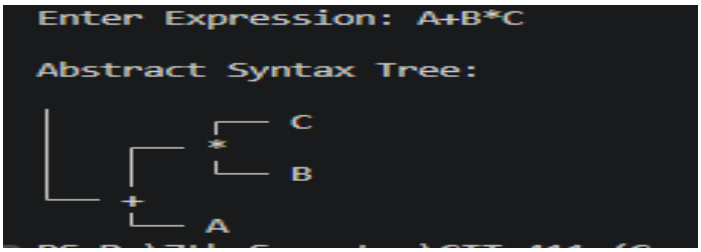
Code:

```
def build_tree(expression):
 expression = expression.replace(" ", "")
# Remove outer parentheses
while expression.startswith("(") and expression.endswith(")"):
 count = 0
 valid = True
for i, ch in enumerate(expression):
if ch == '(':
            count += 1
 elif ch == ')':
             count -= 1
if count == 0 and i != len(expression) - 1:
             valid = False
             break;
 if valid:
 expression = expression[1:-1]
 else:
 break
```

```
# Operator precedence
for operators in ["+-", "*/"]:
count = 0
for i in range(len(expression) - 1, -1, -1):
ch = expression
if ch == ')':
        count += 1
 elif ch == '(':
        count -= 1
elif count == 0 and ch in operators:
         if i == 0 or expression[i - 1] in "+-*/(":
        continue
return [
         ch,
          build_tree(expression[:i]),
          build_tree(expression[i + 1:])
         ]
return expression
def print_tree(tree, prefix="", is_left=True):
 if not isinstance(tree, list):
print(prefix + ("└── " if is_left else "┌── ") + str(tree))
  return
print_tree(
  tree[2],
 prefix + ("│   " if is_left else "    "),
False
```

```
    )
    print(prefix + ("└── " if is_left else "┌── ") + tree[0])
print_tree(
    tree[1],
    prefix + ("    " if is_left else "│   "),
    True
    )
# Main
expression = input("Enter Expression: ")
tree = build_tree(expression)
print("\nAbstract Syntax Tree:\n")
print_tree(tree)
```

Input & Output:



2. Infix to Postfix:

This program converts an infix expression into a postfix expression using a stack. It processes operands directly and uses operator precedence and parentheses to determine the correct order of operators. Finally, it displays the converted postfix expression**.**

Code:

```
def infix_to_postfix(expression):
```

```
stack = []
 postfix = ""
  precedence = {
 '+': 1,
'-': 1,
'*': 2,
'/': 2,
'^': 3
}
 for ch in expression:
 if ch == ' ':
 continue
# Operand
if ch.isalnum():
postfix += ch
 # Opening parenthesis
 elif ch == '(':
 stack.append(ch)
 # Closing parenthesis
 elif ch == ')':
 while stack and stack[-1] != '(':
            postfix += stack.pop(
 if stack:
             stack.pop()
# Operator
 else:
```

```
 while (stack and stack[-1] != '(' and
            precedence.get(stack[-1], 0) >= precedence.get(ch, 0)):
            postfix += stack.pop()
stack.append(ch)
# Pop remaining operators
while stack:
 postfix += stack.pop()
  return postfix
# Main
expression = input("Enter Infix Expression: ")
result = infix_to_postfix(expression)
print("Postfix Expression:", result)
```

Input & Output:

```
Enter Infix Expression: A*B+C
Postfix Expression: AB*C+
```

3. Keyboard Identifier:

This program distinguishes keywords from identifiers using a manually created symbol table. It takes a word as input and checks whether it is a keyword, a valid identifier, or an invalid word. Finally, it displays the classification result.

Code:

```
# Manually created Keyword Symbol Table
symbol_table = [
```

```
 "if", "else", "elif", "while", "for",
"break", "continue", "return", "def",
 "class", "int", "float", "char",
 "void", "import", "from", "as",
 "try", "except", "finally", "True",
    "False", "None", "and", "or", "not"
]
word = input("Enter a word: ")
if word in symbol_table:
 print(word, "is a Keyword")
elif word.isalpha() and not word[0].isdigit():
 print(word, "is an Identifier")
else:
    print(word, "is Invalid")
```

Input & Output:

```
Enter a word: Maynul
Maynul is an Identifier
```

4. Parse_Tree :

This program constructs a Parse Tree from an arithmetic expression using predefined grammar rules. It identifies addition and subtraction operators, recursively divides the expression into smaller parts, and displays the resulting tree structure using a recursive function.

Code:

```
def build_tree(expression):
    expression = expression.replace(" ", "")

    # Find + or - operator
    for i in range(len(expression) - 1, -1, -1):
        if expression[i] in "+-":
            return {
                "list": [
                    build_tree(expression[:i]),
                    expression[i],
                    {"term": expression[i + 1:]}
                ]
            }

    # list -> term
    return {"list": {"term": expression}}

def print_tree(tree, prefix="", is_last=True):
    if isinstance(tree, dict):
        for key, value in tree.items():
            print(prefix + ("└── " if is_last else "├── ") + key)

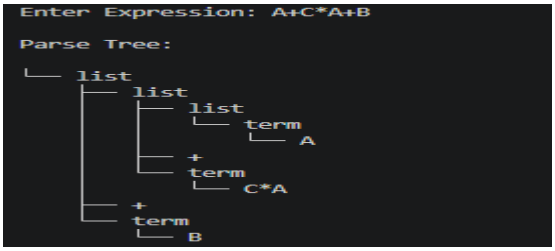
            if isinstance(value, list):
                for i, item in enumerate(value):
                    print_tree(
                        item,
                        prefix + ("    " if is_last else "│   "),
                        i == len(value) - 1
                    )
            else:
                print_tree(value, prefix + ("    " if is_last else "│   "), True)

    else:
        print(prefix + ("└── " if is_last else "├── ") + str(tree))

# Main
expression = input("Enter Expression: ")

tree = build_tree(expression)
print("\nParse Tree:\n")
print_tree(tree)
```

Input & Output:



5. Remove_White_Space :

This program removes unnecessary white spaces and comments from a source code. It scans the code character by character, identifies single-line and multi-line comments, and skips whitespace characters. Finally, it displays the cleaned source code.

CODE:

```
def remove_whitespace_comments(code):
  result = []
 i = 0
 while i < len(code):
  # Remove single-line comments
            if code[i:i+2] == "//":
  while i < len(code) and code[i] != '\n':
              i += 1
            # Remove multi-line comments
 elif code[i:i+2] == "/*":
   i += 2
 while i < len(code) and code[i:i+2] != "*/":
              i += 1
  i += 2
 # Remove white spaces
```

```
  elif code[i].isspace():
  i += 1
  else:
     result.append(code[i])
    i += 1
   return ''.join(result)
code = """
int a = 10;  // variable
int b = 20;
/* This is
a comment */
"""
print("Original Code:")
print(code)
print("After Removing:")
print(remove_whitespace_comments(code))
```

Output:

